SQL and Database Testing

Section A: Conceptual Questions

1. How does understanding database types help decide the testing approach?

Knowing whether the application uses a relational database (MySQL, PostgreSQL, SQL Server) versus a NoSQL database (MongoDB, Cassandra) changes the entire test strategy.

Relational databases enforce schema, data types, and referential integrity through constraints, so testing focuses heavily on:

- Constraint validation (Primary Key, Foreign Key, NOT NULL, UNIQUE, CHECK)
- Transaction / ACID behaviour (atomicity, consistency, isolation, durability)
- Join-based query correctness across normalised tables
- Stored-procedure and trigger validation

NoSQL databases are schema-flexible, so testing instead focuses on:

- Data consistency across distributed nodes
- Document structure validation
- Eventual consistency timing and replication lag

Understanding the database type also determines which SQL dialect and tools to use, how to write validation queries, how to approach performance/load testing, and what kind of defects are likely (e.g., orphaned records in relational DBs vs. duplicate/inconsistent documents in NoSQL). In short, the database type dictates the test design techniques, the queries written, and the risk areas a tester must prioritise.

2. Why is filtering data with conditions more effective than validating random records?

Random record validation only confirms that some data looks correct; it does not guarantee that the underlying business logic is applied consistently across the entire dataset.

Condition-based filtering (e.g., WHERE status='Active' AND signup_date > '2025-01-01') offers several advantages:

- Targets specific business rules, edge cases, and boundary conditions that random sampling is likely to miss
- Is repeatable and directly traceable to requirements, making it auditable
- Surfaces systemic defects (every record matching a condition has the same bug) rather than isolated, one-off issues

- Scales efficiently — a single filtered query can validate thousands of records against a rule in seconds

Random spot-checks would need an impractically large sample size to achieve the same confidence level.

3. How does database validation help find the root cause of incorrect UI data?

When the UI displays incorrect data, there are typically three possible failure points:

- The database itself holds wrong data
- The backend/API is misprocessing correct data
- The front end is misrendering correct data it received

Querying the database directly isolates the first possibility:

- If the database value is already wrong → the defect lies in data entry, a faulty stored procedure, a bad migration script, or an upstream integration.
- If the database value is correct but the UI is wrong → the defect must be in the API layer or the front-end rendering logic.

This systematic elimination — comparing the UI output, API response, and raw DB record side by side — lets a tester pinpoint exactly which layer introduced the defect, speeds up triage, and gives developers an accurate, evidence-backed defect report.

4. When is it acceptable (or risky) to update data directly in the database?

Acceptable when:

- Setting up specific edge-case states (e.g., an expired password, a particular status) that would be slow or impossible to reproduce purely through the UI
- Working in a non-production, isolated test environment
- Validating that the application correctly reads and reacts to a given data state, independent of how that state was created

Risky when:

- It bypasses application business logic and validation rules — the UI/API layer might normally prevent an 'Inactive' user from having a future login date, but a direct DB update can create that inconsistent state
- Done in a shared environment — it can corrupt data that other testers depend on
- Done in production-like environments — it skips audit trails, triggers, and cascades
- It masks real defects, since the tester is manually forcing data the application itself may never legitimately produce

5. How do database constraints reduce defects reaching production?

Constraints enforce data integrity at the lowest possible layer, so invalid data is rejected at the point of entry rather than silently stored and surfacing as a bug later. Key examples:

- NOT NULL — prevents incomplete records
- UNIQUE — prevents duplicate usernames or email addresses
- FOREIGN KEY — prevents orphaned records (e.g., an order referencing a deleted user)
- CHECK / ENUM — prevents out-of-range or invalid status values

Because these rules are enforced regardless of which application, script, or integration writes to the database, they act as a final safety net even when application-level validation has a gap or is bypassed. This significantly reduces the category of defects caused by inconsistent or malformed data, meaning fewer downstream issues in reporting, business logic, and the UI — and fewer production incidents traced back to 'bad data' rather than 'bad code'.

Section B: Practical SQL Tasks

1. Table Creation: Create a Users table with appropriate data types (e.g., INT, VARCHAR, ENUM). Fields must include: User Identifier (Primary Key), Username, Email, Password, and Status.

```
CREATE TABLE Users (
    UserID        INT AUTO_INCREMENT PRIMARY KEY,
    Username      VARCHAR(50)  NOT NULL UNIQUE,
    Email         VARCHAR(100) NOT NULL UNIQUE,
    Password      VARCHAR(255) NOT NULL,
    Status        ENUM('Active','Inactive','Pending') NOT NULL DEFAULT 'Pending',
    LastLoginDate DATE,
    CreatedDate   DATE NOT NULL DEFAULT (CURRENT_DATE)
);
```

2. Data Insertion: Write INSERT INTO statements to add 5 test users. Ensure a mix of data for testing, including at least one "Inactive" status and one user with a last login date older than a year.

```
INSERT INTO Users (Username, Email, Password, Status, LastLoginDate, CreatedDate)
VALUES
('john_doe',   'john.doe@gmail.com',    'Pass@123',   'Active',   '2026-06-15', '2024-01-10'),
('jane_smith', 'jane.smith@yahoo.com',  'Secure@456', 'Active',   '2026-06-18', '2024-02-15'),
('mike_brown', 'mike.brown@gmail.com',  'Mike@789',   'Inactive', '2024-03-01', '2023-05-20'),
('lisa_white', 'lisa.white@outlook.com','Lisa@321',   'Pending',  NULL,         '2026-05-01'),
('alex_jones', 'alex.jones@gmail.com',  'Alex@654',   'Active',   '2025-01-10', '2022-11-11');
```

Dataset notes:

- mike_brown — Inactive status
- lisa_white — Pending status, never logged in (NULL LastLoginDate)
- mike_brown and alex_jones — LastLoginDate older than one year from 2026-06-21

3. Data Retrieval (10 Queries): Write 10 SELECT queries demonstrating:

● Filtering: Using WHERE for specific statuses.

● Pattern Matching: Using LIKE (e.g., finding emails ending in @gmail.com).

● Sorting: Using ORDER BY for usernames or identifiers.

Query 1 — Filtering: All Active Users

```
SELECT * FROM Users WHERE Status = 'Active';
```

Query 2 — Filtering: All Inactive Users

```
SELECT * FROM Users WHERE Status = 'Inactive';
```

Query 3 — Filtering: All Pending Users

```
SELECT * FROM Users WHERE Status = 'Pending';
```

Query 4 — Pattern Matching: Emails Ending in @gmail.com

```
SELECT Username, Email FROM Users WHERE Email LIKE '%@gmail.com';
```

Query 5 — Pattern Matching: Usernames Starting with 'j'

```
SELECT Username FROM Users WHERE Username LIKE 'j%';
```

Query 6 — Sorting: All Users by Username (ASC)

```
SELECT * FROM Users ORDER BY Username ASC;
```

Query 7 — Sorting: All Users by UserID (DESC)

```
SELECT * FROM Users ORDER BY UserID DESC;
```

Query 8 — Conditions (AND): Active Users with Gmail

```
SELECT * FROM Users
WHERE Status = 'Active' AND Email LIKE '%@gmail.com';
```

Query 9 — Conditions (OR): Inactive or Pending Users

```
SELECT * FROM Users
WHERE Status = 'Inactive' OR Status = 'Pending';
```

Query 10 — Conditions (IS NOT NULL): Users Who Have Logged In

```
SELECT Username, LastLoginDate FROM Users
WHERE LastLoginDate IS NOT NULL
ORDER BY LastLoginDate DESC;
```

4. Data Update: Write an UPDATE statement to change the Status of specific users (e.g., setting "Pending" accounts to "Inactive" based on a condition).

Move long-standing Pending accounts (created over 30 days ago with no login) to Inactive:

```
UPDATE Users
SET Status = 'Inactive'
WHERE Status = 'Pending'
  AND LastLoginDate IS NULL
  AND CreatedDate < DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY);
```

5. Data Deletion: Write a DELETE query to remove users who haven't logged in for over a year.

Remove users who haven't logged in for over a year:

```
DELETE FROM Users
WHERE LastLoginDate IS NOT NULL
  AND DATEDIFF(CURRENT_DATE, LastLoginDate) > 365;
```

Section C: Mini Project — End-to-End Database Validation

1. Test Planning Document: Prepare a Test Plan focusing on database testing scope, risks, test data strategy, and validation approach.

Objective

Validate that all backend database operations triggered by the User Management module — registration, login, profile update, and deactivation — correctly persist, update, and enforce data integrity, and that UI/API behaviour matches the underlying database state at all times.

Scope

In scope:

- The Users table and directly related tables (e.g., login history, audit logs)
- Validation of CRUD operations originating from the UI and API
- Constraint and data-type enforcement
- Data consistency between UI, API, and database

Out of scope: database server performance/load testing, network-level security testing, and infrastructure (backup/replication) testing — these are handled by separate efforts.

Risks

- Hidden or undocumented constraints causing unexpected insert/update failures
- Direct database manipulation during testing masking real application defects
- Test data overlapping with other testers' data in a shared environment (false positives/negatives)
- Date/time logic (e.g., 'inactive for a year') behaving inconsistently across time zones
- Sensitive data (passwords, emails) being exposed in logs or query results during testing

Test Data Strategy

- Use a dedicated, isolated test database refreshed from a sanitised baseline before each test cycle
- Maintain a mix of valid, boundary, and invalid data sets (max-length usernames, duplicate emails, null passwords, all three status values)
- Use the UI/API wherever possible to create data; reserve direct SQL inserts only for fast setup of specific edge-case states, clearly documented
- Avoid real personal data; use synthetic but realistic test data

Validation Approach

- Step 1: Perform an action through the UI or API (e.g., register a user)
- Step 2: Query the database directly to confirm the resulting record matches expectations
- Step 3: Cross-check that the UI/API response is consistent with the database state
- Step 4: Use targeted SQL queries (filtering, joins, aggregate counts) rather than full-table review
- Step 5: Log any mismatch as a defect with the UI value, API response, and raw DB value attached as evidence

2. Requirements Traceability Matrix: Create a Requirements Traceability Matrix linking user requirements to database validation queries.

Req ID	User Requirement	Test Case ID	Validation Query	Status
1	User can register with unique username and email	1	SELECT * FROM Users WHERE Username='<new_user>';	**Pass**
2	System rejects duplicate email registration	2	SELECT COUNT(*) FROM Users WHERE Email='<dup>';	**Pass**
3	New users default to Pending status	3	SELECT Status FROM Users WHERE UserID=<id>;	**Pass**
4	User can update profile (email/username)	4	SELECT Email, Username FROM Users WHERE UserID=<id>;	**Pass**
5	Inactive users cannot log in	5	SELECT Status FROM Users WHERE UserID=<id>; cross-check with login API	**Fail**
6	Password must not be stored in plain text	6	SELECT Password FROM Users WHERE UserID=<id>;	**Pass**
7	Users inactive 1+ year are flagged/removed	7	SELECT * FROM Users WHERE DATEDIFF(CURRENT_DATE, LastLoginDate) > 365;	**Pass**
8	Email must follow a valid format	8	SELECT Email FROM Users WHERE Email NOT LIKE '%_@__%.__%';	**Pass**

3. End-to-End Test Scenario: Write one End-to-End test scenario covering:

Scenario ID: E2E-01

Title: User Registration → DB Entry Validation → Profile Update → DB Update Validation

Step	Action	Expected Result	Validation Method	Status
1	Submit valid registration (username, email, password)	Success message displayed on UI	UI observation	Pass
2	Query DB for the new user	New row exists; Status='Pending'; Password is hashed	SELECT * FROM Users WHERE Email='<test_email>';	Pass

3	Update username and email via profile settings	Update accepted; UI shows new values	UI observation	Pass
4	Re-query DB by UserID	Username and Email columns updated; UserID and CreatedDate unchanged	SELECT * FROM Users WHERE UserID=<id>;	Pass
5	Attempt to update email to one already in use	Update rejected with clear error	UI + SELECT COUNT(*) WHERE Email='<dup>';	Pass
6	Compare final UI profile with DB record	All values match exactly	Manual cross-check	Pass

Pass/Fail Criteria: Scenario passes only if every UI-visible value at each step exactly matches the corresponding database record, the duplicate-email update is correctly rejected, and no orphaned or duplicate rows are created.

4. Test Summary Report: Prepare a Test Summary Report including executed queries, defects identified, and final recommendation.

Module: User Management System — Database Layer

Tested By: [Insert Tester Name]

Executed Queries Summary

- 10 SELECT queries — filtering, pattern matching, sorting, AND/OR/IS NOT NULL — all executed successfully
- 1 INSERT batch (5 test users) — executed successfully, constraints respected
- 1 UPDATE query (Pending → Inactive based on age/inactivity) — 1 row affected
- 1 DELETE query (users inactive > 1 year) — reviewed via SELECT first, then committed; 2 rows removed
- E2E scenario E2E-01 fully executed across registration and profile update flows

Defects Identified

Defect ID	Description	Severity	Status	Priority
DEF-01	Inactive users were still able to authenticate via the login API despite Status='Inactive' in the database	**High**	Open	P1 — Blocks Release

DEF-02	Profile update allowed a duplicate email when bypassing client-side validation and hitting the API directly	**Medium**	Open	P2 — Fix Before Release
DEF-03	LastLoginDate was not updated in the DB after a successful login through the mobile client	**Low**	Open	P3 — Next Sprint

Final Recommendation

The core registration and profile-update flows correctly persist and reflect data in the database, and standard CRUD operations, filtering, and constraint enforcement behave as expected. However:

- DEF-01 (High) — Inactive users able to authenticate — represents a security and business-logic risk and should block release until fixed and re-tested.
- DEF-02 (Medium) — Duplicate-email bypass — indicates that uniqueness validation relies on the client rather than being consistently enforced at the API/database layer; add/confirm a UNIQUE constraint enforcement path at the API level.
- DEF-03 (Low) — LastLoginDate not updated on mobile — tracked for a future release.

Recommendation: A fix-and-retest cycle focused on DEF-01 and DEF-02 is required before sign-off. The database layer is otherwise functioning correctly and all SQL queries execute without errors.